

# Software Design Document (SDD)

## Agentic UPI Commerce Bridge — AP2/UCP × Razorpay UPI Autopay

**Document Version:** 1.0

**Date:** 2026-08-28

**Status:** SDD-Ready (Post-QA Consolidation)

**Classification:** Buildathon Submission — Razorpay AI Buildathon, “AI Growth & Agentic Commerce” Track

**Deadline:** September 5, 2026

**Team Size:** Solo Developer + AI Pair-Programming

---

## 1. Introduction

### 1.1 Purpose

This SDD translates the SRS v1.1 consolidated requirements into implementable module designs, interface contracts, database schemas, and deployment topology for the Agentic UPI Commerce Bridge MVP. It is the single source of truth for code generation, CI/CD configuration, and demo execution.

### 1.2 Scope

- **In Scope (MVP):** Modules 1–8 running under Docker Compose, Razorpay Test Mode, one Shopify sandbox merchant, up to 3 parallel negotiation sub-agents (demo), one gracefully handled failure on camera.
- **Out of Scope (Documented for Production):** Kubernetes, HSM/KMS, Kafka/Redis Streams, full 10-merchant parallel scale, 500-vector injection benchmark, external red-team engagement, multi-year audit retention.

### 1.3 Design Principles

1. **Deterministic Sandwich:** Probabilistic AI reasoning is strictly bounded between deterministic constraint compilation and deterministic guardrail validation.
2. **Fail Closed:** Any dependency unavailability, schema violation, or grounding failure results in rejection or HITL escalation — never silent pass-through.
3. **Explainability:** Every money-moving action is traceable from human intent → LLM proposal → guardrail decision → signed mandate → settlement → audit record within 30 seconds.
4. **Zero Trust in LLM:** The LLM has no signing keys, no Razorpay credentials, no direct payment execution capability, and no access to the Mandate Vault under any configuration (INV-001).

### 1.4 Glossary & References

- **SRS v1.1:** Software Requirements Specification (consolidated from six role perspectives).
- **AP2:** Agent Payments Protocol (Google → FIDO Alliance).
- **UCP:** Universal Commerce Protocol (Google + Shopify/Etsy).
- **A2A:** Agent-to-Agent Protocol (Google → Linux Foundation).
- **MCP:** Model Context Protocol (Anthropic).
- **RFC 8785:** JSON Canonicalization Scheme (JCS).
- **JWS:** JSON Web Signature (RFC 7515).

- **ES256:** ECDSA using P-256 and SHA-256.
- **HITL:** Human-in-the-Loop.
- **SSOT:** Single Source of Truth.

## 2. System Architecture Overview

### 2.1 The Deterministic Sandwich

|                                                                                                    |
|----------------------------------------------------------------------------------------------------|
| L8: Human Delegation & Identity<br>(WebAuthn/Passkey, Constraint Compiler)                         |
| L7: Probabilistic AI Reasoning<br>(GPT-4o via MCP, parallel discovery, negotiation sub-agents)     |
| L6: Deterministic Validation<br>(Guardrail Shell: Schema → Policy → Grounding → Confidence → HITL) |
| L5: Agent Communication & Discovery<br>(A2A Agent Cards/Tasks, UCP Discovery)                      |
| L4: Trust & Authorization<br>(AP2 Mandates: Intent → Cart → Payment, Mandate Vault JWS signing)    |
| L3: Commerce & Checkout<br>(UCP state machine, cart locking, parallel negotiation)                 |
| L2: Payment Orchestration<br>(Razorpay UPI Autopay, SPTs, conditional escrow)                      |
| L1: Transport & Governance<br>(HTTP/1.1 REST, TLS 1.3, OAuth2/mTLS, PCI-DSS aspirational, RBI)     |

### 2.2 Trust Zones & Network Segmentation

| Zone                    | Contains                                                                 | Can Reach                                                       |
|-------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------|
| Untrusted Ingestion     | A2A Agent Cards, UCP manifests, Shopify webhooks, product listings       | Only Schema/Signature Verifier                                  |
| LLM Context             | Probabilistic Reasoning Core, negotiation sub-agents                     | Only validated structured context; output = ProposalObject only |
| Deterministic Guardrail | Guardrail Shell (Schema, Policy, Grounding, Confidence, Session Manager) | ProposalObject + signed manifests → Mandate Vault               |
| Signing Zone            | Mandate Vault                                                            | Nothing externally; only signs what Guardrail approves          |
| Settlement Zone         | UPI Payment Adapter                                                      | Only Razorpay API (api.razorpay.com)                            |

Table 1 – continued

| Zone               | Contains                   | Can Reach                                                                    |
|--------------------|----------------------------|------------------------------------------------------------------------------|
| <b>Ledger Zone</b> | PostgreSQL + Ledger Writer | Read by most; written only by Ledger Writer + independent component outboxes |

**Critical Invariant:** The LLM container has **no network route** to the Mandate Vault, Payment Adapter, or Razorpay API. Verified by automated smoke test: `curl/nc` from Reasoning Core to Vault times out with “No route to host.”

## 2.3 Docker Compose Network Topology

```
# docker-compose.yml — Network Definitions (MVP)
networks:
  net-untrusted:
    driver: bridge
  net-llm:
    driver: bridge
    internal: true          # No external egress
  net-guardrail:
    driver: bridge
    internal: true
  net-signing:
    driver: bridge
    internal: true
  net-settlement:
    driver: bridge
    internal: false        # Required for Razorpay egress
  net-ledger:
    driver: bridge
    internal: true
```

### Service Network Attachments:

- `constraint-compiler` → `net-untrusted`, `net-guardrail`
- `negotiation-engine` → `net-llm`, `net-guardrail`
- `guardrail-shell` → `net-guardrail`, `net-signing`, `net-ledger`
- `mandate-vault` → `net-signing` only
- `upi-payment-adapter` → `net-signing`, `net-settlement`, `net-ledger`
- `universal-commerce-adapter` → `net-untrusted`, `net-guardrail`
- `ledger` (PostgreSQL) → `net-ledger`
- `jaeger` → `net-guardrail`, `net-ledger` (UI exposed on host port 16686)

### 3. Module Design

#### 3.1 Canonical Module Numbering (Frozen)

| Module | Component                    | Stack                            | MVP Deployable                                     |
|--------|------------------------------|----------------------------------|----------------------------------------------------|
| 1      | Constraint Compiler          | Python<br>(FastAPI)              | constraint-compiler service                        |
| 2      | Probabilistic Reasoning Core | Python (OpenAI SDK)              | Embedded within negotiation-engine                 |
| 3      | Negotiation Engine           | Node.js<br>(Express)             | negotiation-engine service                         |
| 4      | Guardrail Shell              | Python<br>(FastAPI)              | guardrail-shell service (includes Session Manager) |
| 5      | Mandate Vault                | Python<br>(FastAPI)              | mandate-vault service                              |
| 6      | UPI Payment Adapter          | Python<br>(FastAPI)              | upi-payment-adapter service                        |
| 7      | Universal Commerce Adapter   | Python<br>(FastAPI)              | universal-commerce-adapter service                 |
| 8      | External Persistent Ledger   | PostgreSQL 15 +<br>Python Writer | ledger service (Postgres) +<br>ledger-writer       |

#### 3.2 Module 1 — Constraint Compiler

**Responsibility:** Transform natural-language purchase intent into a canonical, hashed, machine-checkable CompiledConstraints object.

**Implementation:**

- **Framework:** FastAPI + Pydantic v2
- **LLM Backend:** GPT-4o-2024-08-06 via OpenAI API with Structured Outputs (response\_format={"type": "json\_schema", ...})
- **Canonicalization:** rfc8785 Python library (RFC 8785 compliant) → SHA-256 hash
- **Latency Budget:** < 1ms for canonicalization + hashing; < 3s end-to-end for structured generation

**API Contract:**

```
POST /v1/compile
Content-Type: application/json

{
  "natural_language_intent": "Buy noise-cancelling headphones under ₹5000 from Amazon or Flipkart within 24h"
}

Response 200:
{
  "compiled_constraints": {
    "max_spend": 5000.00,
    "currency": "INR",
```

```

{
  "merchant_scope": ["amazon.in", "flipkart.com"],
  "category": "electronics/headphones",
  "validity_window_hours": 24,
  "refundability_required": false,
  "hard_exclusions": ["wired_only"]
},
{
  "constraint_hash": "sha256:abc123...",
  "canonical_json": "{\"category\":\"electronics/headphones\",...}\" // RFC 8785
}

```

#### Key Decisions:

- Soft preferences (e.g., “prefer wireless”) are kept in a separate `soft_preferences` field and NOT enforced by the Policy Engine (FR-CC-003).
- The `constraint_hash` is carried through every downstream mandate and audit record (FR-CC-002).

### 3.3 Module 2 — Probabilistic Reasoning Core

**Responsibility:** LLM-driven negotiation, ranking, and proposal generation. Never executes anything.

#### Implementation:

- **Backend:** GPT-4o-2024-08-06 (locked for MVP)
- **Access:** Via MCP server interface inside the Negotiation Engine container
- **Output:** Strictly `ProposalObject` (Appendix C.1) via constrained decoding
- **Tool Choice:** `tool_choice="none"` — no direct tool/API calls permitted

#### Confidence Formula (FR-GRD-006):

$C = 0.40 \times S_{\text{logprob}} + 0.40 \times S_{\text{grounding}} + 0.20 \times S_{\text{schema}}$   
 Threshold:  $C \geq 0.85$  passes;  $C < 0.85$  escalates to HITL.

**$S_{\text{logprob}}$  Calculation:** Geometric mean of linear probabilities for tokens in `selected_candidate_id` and `offer_price`:

$$S_{\text{logprob}} = (\prod \exp(\text{logprob}_i))^{(1/k)} \quad \text{for } i = 1..k$$

**Fallback (FR-GRD-006a):** If logprobs unavailable, N=3 self-consistency voting:

- Unanimous (3/3):  $S_{\text{consensus}} = 1.0$
- Majority (2/3):  $S_{\text{consensus}} = 0.66$
- Disagreement (1/3):  $S_{\text{consensus}} = 0.0$
- Formula shifts to:  $C = 0.50 \times S_{\text{consensus}} + 0.30 \times S_{\text{grounding}} + 0.20 \times S_{\text{schema}}$

### 3.4 Module 3 — Negotiation Engine

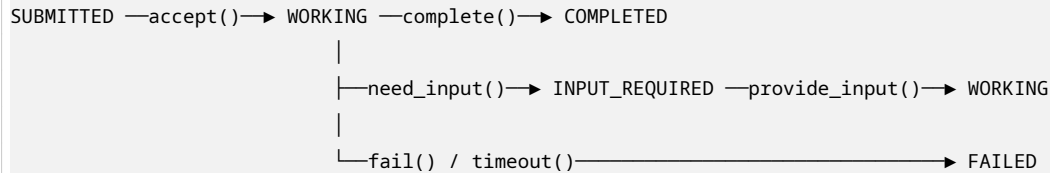
**Responsibility:** Orchestrate multi-round, multi-merchant negotiation with monotonicity checks and round limits.

#### Implementation:

- **Framework:** Node.js + Express

- **Concurrency:** `Promise.allSettled` with 30s per-round timeout (FR-NEG-002)
- **Sub-Agents:** 3 for demo (up to 10 architecturally); each runs in an isolated `asyncio` context via `ProcessPoolExecutor` (capped at 10 workers)
- **State Persistence:** PostgreSQL `a2a_tasks` table (MVP); Redis Streams for production

#### A2A Task State Machine:



#### Monotonicity Check (FR-NEG-003):

```

# Programmatic evaluation, never trusting LLM explanation
if current_offer_price > previous_offer_price:
    terminate_negotiation(merchant_id, reason="MONOTONICITY_VIOLATION")

```

#### Circuit Breaker (FR-NEG-006):

- Open after: P95 > 2000ms (30s window) OR error rate > 20% (1min)
- Open duration: 60s, single half-open probe
- Fallback: Cached manifest (max staleness 5min) or HITL `DISCOVERY_UNAVAILABLE`

### 3.5 Module 4 — Guardrail Shell

**Responsibility:** The only path between Probabilistic Core and Mandate Vault. Enforces schema, policy, grounding, confidence, and HITL escalation.

#### Implementation:

- **Framework:** FastAPI + Pydantic
- **Sub-Modules:**
  1. **SchemaValidator:** Pydantic validation against `ProposalObject` schema (Appendix C.1). Max 2 retries before HITL (FR-GRD-003).
  2. **PolicyEngine:** Plain-code hard-constraint checks (amount `max_spend`, merchant in allowlist, validity window, refundability).
  3. **GroundingOracle:** Binary gate — exact field matching against signed UCP manifest Redis cache. < 2ms lookup. Zero tolerance for hallucinated facts.
  4. **Confidence/Escalation Gate:** Computes C per FR-GRD-006. Hard escalations (FR-GRD-007): out-of-scope merchant, amount cap exceeded, expired/revoked mandate, grounding failure.
  5. **SessionManager:** APScheduler `DateTrigger` for UCP checkout TTLs (§4.2.3.1). Owns timeout enforcement and audit event generation.

#### Execution Pipeline (NFR-PERF-001, p95 < 50ms):

```

1. Schema Validator      (~1ms)  → reject malformed immediately
2. Policy Engine         (~5ms)  → hard constraint checks

```

|                     |                                                           |
|---------------------|-----------------------------------------------------------|
| 3. Grounding Oracle | (~20ms) → parallel Redis lookups (price, stock, identity) |
| 4. Confidence Gate  | (~2ms) → score computation                                |
| 5. HITL Escalation  | (~5ms) → payload assembly if C < 0.85                     |

**Fail-Closed Behavior (FR-GRD-009):** If Policy Engine, Grounding Oracle, or Mandate Vault is unreachable → REJECT or HITL. Never pass-through on timeout.

**UCP Checkout Session TTLs (§4.2.3.1):**

| State                | TTL   | Fallback on Timeout                                           |
|----------------------|-------|---------------------------------------------------------------|
| incomplete           | 6h    | Auto-cancel SESSION_EXPIRED                                   |
| requires_escalation  | 24h   | Auto-cancel HITL_TIMEOUT                                      |
| ready_for_complete   | 15min | Release cart lock / auto-cancel                               |
| complete_in_progress | 5min  | Retry Razorpay status → payment_pending_reconciliation + HITL |

**Outbox Pattern (Guardrail → Vault handoff):** The Guardrail writes an approval record to PostgreSQL vault\_outbox in the same transaction as the guardrail decision. A background poller forwards to the Vault. Guarantees zero orphaned signatures.

### 3.6 Module 5 — Mandate Vault

**Responsibility:** The only component with private-key access. Signs AP2 mandates as JWS (ES256).

**Implementation:**

- **Framework:** FastAPI
- **Crypto Libraries:** rfc8785 (canonicalization) + jwcrypto (JWS signing/verification)
- **Algorithm Allowlist:** {"ES256"} for AP2 mandates; {"Ed25519"} for internal identity (FR-MV-003)
- **Key Partitioning (FR-MV-004):**
  1. ap2\_signing\_key — ES256 mandate signing
  2. agent\_identity\_key — Ed25519 internal identity
  3. ledger\_integrity\_key — checkpoint signing
  4. razorpay\_operation\_credential — Razorpay API auth (technically a credential, not a signing key)

**Security Hardening:**

```
# docker-compose.yml — Mandate Vault
services:
  mandate-vault:
    user: "10001:10001"
    read_only: true
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true
    tmpfs:
```

```

- /tmp
networks:
- net-signing
secrets:
- ap2_signing_key
- agent_identity_key

```

### API Contract (Guardrail → Vault):

```

POST /v1/mandate/sign
Content-Type: application/json
X-Client-Cert-CN: guardrail-shell

{
  "mandate_type": "INTENT || CART || PAYMENT",
  "payload_canonical_json": "string", // RFC 8785 bytes
  "expected_canonical_sha256": "sha256:...",
  "key_id": "2026-08-ap2-1",
  "guardrail_decision_id": "uuid",
  "constraint_hash": "sha256:..."
}

Response 200:
{
  "mandate_id": "uuid",
  "compact_jws": "eyJhbGciOiJIUzI1NiJ9...",
  "kid": "2026-08-ap2-1",
  "canonical_sha256": "sha256:..."
}

```

### Vault Security Rules:

1. Authenticate caller via mTLS client identity
2. Verify guardrail\_decision\_id corresponds to an approved decision
3. Revalidate mandate type/schema/expiry/merchant/amount
4. Canonicalize mandate\_json using RFC 8785 internally
5. Verify expected\_canonical\_sha256
6. Verify key\_id is authorized for the artifact type
7. Sign **only** the canonicalized payload
8. **Never** expose generic sign(bytes) functionality (prevents signing oracle attacks)

### 3.7 Module 6 — UPI Payment Adapter

**Responsibility:** Bridge signed AP2 Payment Mandates to Razorpay UPI Autopay S2S API.

#### Implementation:

- **Framework:** FastAPI
- **Razorpay Endpoints:**



- POST /customers
- POST /orders (with token={})
- POST /payments/create/upi
- POST /payments/create/recurring
- **Mode:** Test Mode ONLY (rzp\_test\_\* keys enforced at startup). Live credentials (rzp\_live\_\*) are rejected with fatal error.

AP2 → Razorpay State Mapping:

| AP2 Mandate State                    | Ledger State                 | Razorpay Token State   | Notes                           |
|--------------------------------------|------------------------------|------------------------|---------------------------------|
| Intent signed                        | INTENT_RECORDED              | —                      | No money-moving capability      |
| Cart signed                          | CART_APPROVED                | —                      | Grounded against merchant offer |
| Payment signed, registration pending | PAYMENT_PENDING_REGISTRATION | intented               | Awaiting UPI app approval       |
| Customer approves in UPI app         | PAYMENT_ACTIVE               | authenticated → active | Debits may now occur            |
| Debit executed                       | SETTLED (per-cycle)          | active                 | One record per successful debit |
| User/system revokes                  | REVOKED                      | revoked                | AFA required (FR-PAY-004)       |
| Validity window elapses              | EXPIRED                      | expired/inactive       | No further debits               |

Debit Atomicity (INV-003 / INV-004):

```
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT state, max_amount, expire_at FROM mandates WHERE id = $1 FOR UPDATE;
-- If state != 'PAYMENT_ACTIVE' or expired → ROLLBACK, return MANDATE_REVOKED/EXPIRED
INSERT INTO debits (mandate_id, idempotency_key, amount, status)
VALUES ($1, $2, $3, 'PENDING')
ON CONFLICT (mandate_id, idempotency_key) DO NOTHING;
-- If no row inserted → return original result (idempotent)
COMMIT;
```

Graceful Failure Handling:

- **Razorpay timeout/ambiguous response:** Mark PENDING, escalate to HITL (FR-PAY-005). Retry with exponential backoff (immediate, 5s, 15s). Max 3 retries.
- **Revocation racing debit:** SELECT ... FOR UPDATE ensures revocation wins (FR-MAP-003).

### 3.8 Module 7 — Universal Commerce Adapter

**Responsibility:** Poll Shopify for inventory/price and expose UCP-compatible manifests.

**Implementation:**

- **Framework:** FastAPI
- **Shopify API:** REST Admin API (MVP); GraphQL facade for production
- **Polling:** 5s intervals when negotiation is active; 60s when idle (FR-UCA-002)
- **Change Detection:** updated\_at comparison + best-effort webhook consumption (FR-UCA-003)
- **Queue:** Bounded queue (capacity 1000); overflow drops oldest + triggers full resync (FR-UCA-004)
- **Webhook Verification:** HMAC-SHA256 signature check before processing (FR-EXT-007)

**IInventoryProvider Protocol (Python):**

```
from typing import Protocol, List, Optional
from dataclasses import dataclass
from decimal import Decimal
from datetime import datetime

@dataclass(frozen=True)
class Product:
    product_id: str
    name: str
    price: Decimal          # Never float. Decimal for INR.
    currency: str           # "INR" for MVP
    available_quantity: int
    merchant_id: str
    updated_at: datetime

@dataclass(frozen=True)
class MerchantCatalog:
    merchant_id: str
    products: List[Product]
    last_synced_at: datetime

class IInventoryProvider(Protocol):
    async def get_product(self, product_id: str) -> Optional[Product]: ...
    async def list_products(self, merchant_id: str) -> List[Product]: ...
    async def check_availability(self, product_id: str, quantity: int) -> bool: ...
    async def get_price(self, product_id: str) -> Decimal: ...
    async def get_catalog(self, merchant_id: str) -> MerchantCatalog: ...
```

**Cart Locking (FR-UCA-007):**

- Redis SET cart\_lock:{cart\_id} {buyer\_id} NX PX 900000 (15min TTL)

- Lock released on: timeout, cancellation, or successful checkout
- Pre-checkout re-verification: Price discrepancy >  $\pm 5\%$  blocks automatic checkout (FR-UCA-005)

### 3.9 Module 8 — External Persistent Ledger

**Responsibility:** SSOT for mandate/revocation state and append-only audit trail.

**Implementation:**

- **Database:** PostgreSQL 15
- **Isolation:** REPEATABLE READ default transaction level
- **Permissions (DR-002 / INV-006):**
  - ledger\_writer → INSERT only on audit + mandate tables
  - guardrail\_reader, adapter\_reader → SELECT only
  - **No application role has UPDATE or DELETE on audit tables**

**Hash Chaining (DR-003):**

```
def insert_audit_event(event):
    last_hash = db.query("SELECT hash FROM audit_events ORDER BY id DESC LIMIT 1")
    previous_hash = last_hash or "0" * 64
    payload_hash = sha256(rfc8785.dumps(event)).hexdigest()
    current_hash = sha256((previous_hash + payload_hash).encode()).hexdigest()
    db.insert(event_id, source, type, payload, previous_hash, current_hash)
```

**Independent Writes (DR-006):** Each component (Guardrail, Vault, Payment Adapter) writes its own audit events independently. No single component can suppress another's log.

## 4. Database Schema (PostgreSQL DDL)

```
-- =====
-- Agentic UPI Commerce Bridge — MVP Database Schema
-- PostgreSQL 15 | SERIALIZABLE isolation on critical paths
-- =====

-- 1. Mandates Table (SSOT for mandate lifecycle)
CREATE TABLE mandates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    state VARCHAR(30) NOT NULL CHECK (state IN (
        'INTENT_RECORDED', 'CART_APPROVED', 'PAYMENT_PENDING_REGISTRATION',
        'PAYMENT_ACTIVE', 'SETTLED', 'REVOKED', 'EXPIRED'
    )),
    mandate_type VARCHAR(20) NOT NULL CHECK (mandate_type IN ('INTENT', 'CART', 'PAYMENT')),
    constraint_hash TEXT NOT NULL,
    max_amount DECIMAL(12,2) NOT NULL,
    merchant_scope JSONB NOT NULL DEFAULT '[]',
    validity_window_hours INT NOT NULL DEFAULT 24,
    expire_at TIMESTAMP NOT NULL,
```

```

    token_id VARCHAR(255),
    buyer_id VARCHAR(255),
    merchant_id VARCHAR(255),
    parent_mandate_id UUID REFERENCES mandates(id),
    signed_jws TEXT,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_mandates_state ON mandates(state);
CREATE INDEX idx_mandates_expire_at ON mandates(expire_at);
CREATE INDEX idx_mandates_buyer ON mandates(buyer_id);

-- 2. Debits Table (Idempotency + Settlement)
CREATE TABLE debits (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    mandate_id UUID NOT NULL REFERENCES mandates(id),
    idempotency_key VARCHAR(255) NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    currency VARCHAR(3) NOT NULL DEFAULT 'INR',
    razorpay_payment_id VARCHAR(255),
    razorpay_order_id VARCHAR(255),
    status VARCHAR(20) NOT NULL CHECK (status IN ('PENDING', 'SUCCESS', 'FAILED', 'RECONCILING')),
    retry_count INT NOT NULL DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),
    UNIQUE(mandate_id, idempotency_key)
);

CREATE INDEX idx_debits_mandate ON debits(mandate_id);
CREATE INDEX idx_debits_status ON debits(status);

-- 3. Audit Events Table (Append-Only, Hash-Chained)
CREATE TABLE audit_events (
    id BIGSERIAL PRIMARY KEY,
    event_id UUID NOT NULL UNIQUE,
    source_component VARCHAR(50) NOT NULL,
    event_type VARCHAR(50) NOT NULL,
    mandate_id UUID,
    transaction_id UUID,
    constraint_hash TEXT,
    llm_invocation_id UUID,
    payload JSONB NOT NULL,
    previous_hash TEXT NOT NULL,
    hash TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Prevent UPDATE/DELETE at application layer (enforced by DB permissions)
CREATE INDEX idx_audit_mandate ON audit_events(mandate_id);
CREATE INDEX idx_audit_event_id ON audit_events(event_id);

```

```

CREATE INDEX idx_audit_created ON audit_events(created_at);

-- 4. Checkout Sessions (UCP State Machine)
CREATE TABLE checkout_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  buyer_id VARCHAR(255) NOT NULL,
  cart_id UUID,
  state VARCHAR(30) NOT NULL CHECK (state IN (
    'incomplete', 'requires_escalation', 'ready_for_complete',
    'complete_in_progress', 'completed', 'cancelled'
  )),
  locked_at TIMESTAMP,
  expires_at TIMESTAMP NOT NULL,
  hitl_payload JSONB,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_checkout_state ON checkout_sessions(state);
CREATE INDEX idx_checkout_expires ON checkout_sessions(expires_at);

-- 5. A2A Tasks
CREATE TABLE a2a_tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  task_id VARCHAR(255) NOT NULL UNIQUE,
  status VARCHAR(20) NOT NULL CHECK (status IN ('SUBMITTED', 'WORKING', 'INPUT_REQUIRED', 'COMPLETED',
    ↪ 'FAILED')),
  merchant_id VARCHAR(255),
  proposal JSONB,
  rejection_context JSONB,
  retry_count INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_a2a_status ON a2a_tasks(status);

-- 6. Vault Outbox (Transactional Outbox Pattern)
CREATE TABLE vault_outbox (
  id BIGSERIAL PRIMARY KEY,
  guardrail_decision_id UUID NOT NULL,
  mandate_type VARCHAR(20) NOT NULL,
  payload JSONB NOT NULL,
  constraint_hash TEXT NOT NULL,
  processed_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_outbox_unprocessed ON vault_outbox(processed_at) WHERE processed_at IS NULL;

-- 7. Revoked Keys (Identity Verification)

```

```

CREATE TABLE revoked_keys (
    kid VARCHAR(255) PRIMARY KEY,
    revoked_at TIMESTAMP DEFAULT NOW(),
    reason TEXT
);

-- 8. Constraint Enforcement Audit (Protocol Mapping)
CREATE TABLE constraint_enforcement_audit (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    timestamp TIMESTAMP DEFAULT NOW(),
    mandate_id UUID,
    transaction_id UUID,
    original_ap2_field TEXT NOT NULL,
    original_ap2_value JSONB,
    upi_capability TEXT NOT NULL,
    downgrade_action VARCHAR(20) NOT NULL CHECK (downgrade_action IN ('DEGRADE', 'ESCALATE', 'REJECT')),
    chosen_replacement_value JSONB,
    reason TEXT NOT NULL,
    enforced_by VARCHAR(50) NOT NULL,
    constraint_hash TEXT NOT NULL,
    srs_reference VARCHAR(50) NOT NULL
);

-- Role-Based Permissions (INV-006 enforcement)
CREATE ROLE ledger_writer;
CREATE ROLE guardrail_reader;
CREATE ROLE adapter_reader;

GRANT INSERT ON audit_events TO ledger_writer;
GRANT INSERT ON mandates TO ledger_writer;
GRANT INSERT ON debits TO ledger_writer;
GRANT INSERT ON constraint_enforcement_audit TO ledger_writer;

GRANT SELECT ON mandates TO guardrail_reader;
GRANT SELECT ON debits TO guardrail_reader;
GRANT SELECT ON audit_events TO guardrail_reader;

GRANT SELECT ON mandates TO adapter_reader;
GRANT SELECT ON debits TO adapter_reader;

-- Explicitly deny UPDATE/DELETE on audit_events for all app roles
-- (Application code must use these roles; superuser is for schema migrations only)

```

---

## 5. Interface Contracts

### 5.1 Inter-Service Transport (MVP)

**Decision:** HTTP/1.1 REST + JSON with OpenAPI 3.0 spec as SSOT. gRPC/Protobuf documented for production evolution.

### Rationale:

- Eliminates protoc toolchain friction for 8-day solo build
- OpenAPI provides equivalent interface contract enforcement
- gRPC is tagged [Production] in SRS; REST is explicitly permitted as MVP fallback

**Canonical JSON Payload:** All structured objects crossing service boundaries are RFC 8785 canonicalized before hashing. The canonical JSON string is transmitted as the body payload with a `Content-Digest: sha256=...` header verified by the receiver.

## 5.2 Negotiation Engine → Guardrail Shell

```
POST /v1/guardrail/validate
Content-Type: application/json
Traceparent: 00-{trace_id}-{span_id}-01

{
  "proposal": {
    "selected_candidate_id": "uuid",
    "offer_price": 4999.00,
    "reasoning": "Best price within constraints",
    "negotiation_round": 1
  },
  "constraint_hash": "sha256:...",
  "metadata": {
    "llm_invocation_id": "uuid",
    "model_name": "gpt-4o-2024-08-06",
    "temperature": 0.0,
    "execution_latency_ms": 242.5
  }
}

Response 200:
{
  "decision": "APPROVED | REJECTED | ESCALATED",
  "confidence_score": 0.92,
  "guardrail_checks": {
    "schema_valid": true,
    "policy_passed": true,
    "grounding_status": "FULLY_GROUNDED",
    "monotonicity_ok": true
  },
  "rejection_context": null, // populated if REJECTED
  "hitl_payload": null      // populated if ESCALATED
}
```

## 5.3 Guardrail Shell → Mandate Vault

```
POST /v1/mandate/sign
Content-Type: application/json
X-Client-Cert-CN: guardrail-shell
```

```
{
  "mandate_type": "PAYMENT",
  "payload_canonical_json": "{...}",
  "expected_canonical_sha256": "sha256:...".
  "key_id": "2026-08-ap2-1",
  "guardrail_decision_id": "uuid",
  "constraint_hash": "sha256:...".
  "expires_at_unix": 1724851200
}

Response 200:
{
  "mandate_id": "uuid",
  "compact_jws": "eyJhbGciOiJIUzI1NiJ9...".
  "kid": "2026-08-ap2-1",
  "canonical_sha256": "sha256:...".
}
```

5.4 Mandate Vault → UPI Payment Adapter

```
POST /v1/payment/execute
Content-Type: application/json
X-Idempotency-Key: {idempotency_key}

{
  "mandate_id": "uuid",
  "signed_payment_mandate_jws": "eyJhbGciOiJIUzI1NiJ9...".
  "idempotency_key": "string",
  "amount_paise": 499900,
  "currency": "INR",
  "constraint_hash": "sha256:...".
}

Response 200:
{
  "razorpay_payment_id": "pay_...",
  "razorpay_order_id": "order_...",
  "status": "SUCCESS | PENDING | FAILED",
  "settled_at": "2026-08-28T21:38:00Z"
}
```

5.5 Error Codes (SRS §6.5 Graceful Failures)

| Failure Mode             | HTTP Status | Error Code            | Returned By                          |
|--------------------------|-------------|-----------------------|--------------------------------------|
| Unauthorized merchant    | 403         | UNAUTHORIZED_MERCHANT | Guardrail Shell                      |
| Amount above mandate cap | 400         | AMOUNT_EXCEEDED       | Guardrail Shell /<br>Payment Adapter |
| Expired mandate          | 403         | MANDATE_EXPIRED       | Guardrail Shell                      |



Table 5 – continued

| Failure Mode                      | HTTP Status | Error Code             | Returned By                   |
|-----------------------------------|-------------|------------------------|-------------------------------|
| Revoked mandate (race won)        | 403         | MANDATE_REVOKED        | Payment Adapter               |
| Malformed/schema-invalid proposal | 400         | SCHEMA_VIOLATION       | Guardrail Shell               |
| Grounding failure                 | 400         | GROUNDING_FAILURE      | Guardrail Shell               |
| Duplicate/replayed request        | 409         | DUPLICATE_REQUEST      | Payment Adapter               |
| Razorpay timeout/ambiguous        | 202         | PENDING_RECONCILIATION | Payment Adapter (+ HITL flag) |

## 6. Protocol-to-Settlement Mapping

### 6.1 AP2 → UPI Autopay Constraint Matrix

| AP2 Constraint            | UPI Representation               | Cryptographically Enforced?          | Policy Engine Enforced?      | Downgrade Action           |
|---------------------------|----------------------------------|--------------------------------------|------------------------------|----------------------------|
| amount                    | token.max_amount (paise)         | No (Razorpay enforces at settlement) | Yes (pre-debit check)        | REJECT if > cap            |
| merchant_scope            | Order metadata only              | No                                   | Yes (allowlist check)        | REJECT if not in scope     |
| expiry                    | token.expire_at                  | Yes (signed into JWS)                | Yes (timestamp check)        | REJECT if expired          |
| frequency                 | NPCI enum (daily/weekly/monthly) | Partial (token binds enum)           | Yes (enum membership)        | DEGRADE to nearest + audit |
| refundability_requirement | Native support                   | No                                   | No                           | ESCALATE to HITL           |
| partial_fulfillment       | Single-block debit only          | No                                   | Yes (block partial mandates) | REJECT for MVP             |
| billing_cycle_cap         | NPCI max 1 debit/cycle           | No                                   | Yes (ledger tracking)        | REJECT if cycle debited    |

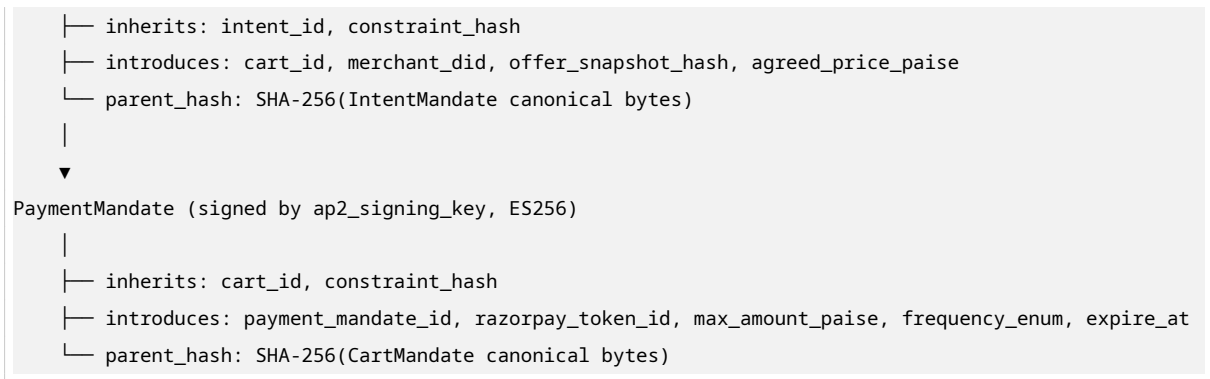
**Rule (INV-007 / FR-MAP-002):** Any constraint that UPI cannot natively enforce is either explicitly downgraded (with audit), escalated to HITL, or rejected. Silent dropping is a Sev-1 defect.

### 6.2 Mandate Artifact Lifecycle

```

IntentMandate (signed by buyer identity key, ES256)
|
|— contains: intent_id, constraint_hash, buyer_did, validity_window
|
▼
CartMandate (signed by ap2_signing_key, ES256)
|

```



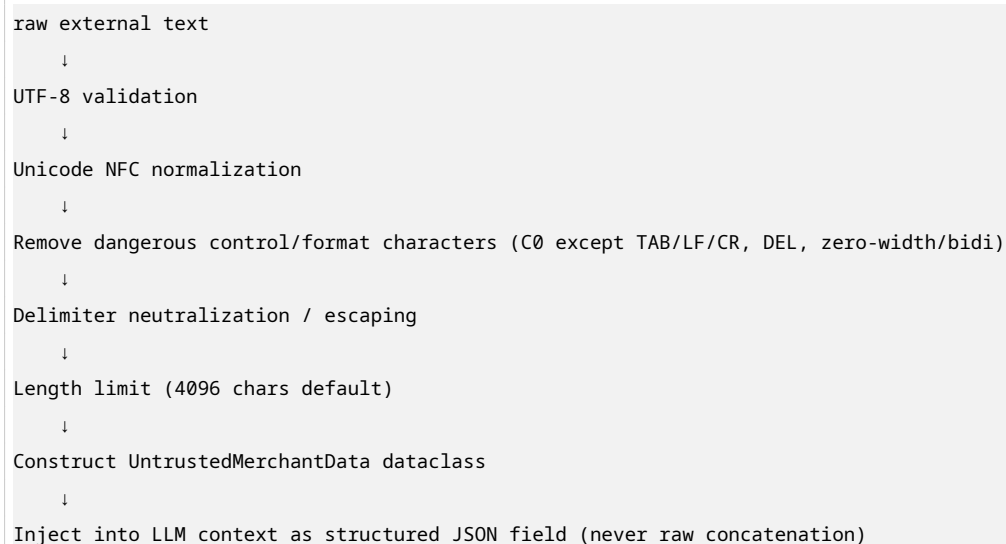
**Binding:** intent\_id → cart\_id → payment\_mandate\_id. Each downstream artifact includes the parent’s hash, creating a cryptographic chain.

---

## 7. Security Architecture

### 7.1 Prompt Injection Defense (SEC-PI-001–005)

#### Sanitization Pipeline:



**Security Oracle:** Injection success is determined by **backend behavior**, not model text:

- Unauthorized policy mutation → FAIL
- Vault signing request without Guardrail approval → FAIL
- Payment API call without Guardrail approval → FAIL
- Secret access attempt → FAIL
- Audit suppression → FAIL

An injection is only “blocked” if the Guardrail Shell explicitly rejects it with `POLICY_VIOLATION` or `GROUNDING_FAILURE`.

### 7.2 Key Management (FR-MV-004 / FR-MV-005)

| Key                  | Purpose                   | MVP Storage                     | Access                      |
|----------------------|---------------------------|---------------------------------|-----------------------------|
| AP2 signing key      | ES256 mandate signing     | Docker secret → Vault only      | Vault container (UID 10001) |
| Agent identity key   | Ed25519 internal identity | Docker secret → Vault only      | Vault container             |
| Ledger integrity key | Checkpoint signing        | Docker secret → Ledger writer   | Ledger writer container     |
| Razorpay credential  | API authentication        | Docker secret → Payment Adapter | Payment Adapter container   |

#### Isolation:

- Vault container: `cap_drop: [ALL]`, `read_only: true`, `no-new-privileges`, dedicated secret mount
- Reasoning Core container: **zero secret mounts**, non-root user
- mTLS required at Vault boundary (SEC-NET-003)

### 7.3 mTLS Configuration (MVP)

**Topology:** Local mini-CA + OpenSSL-generated certificates + native gRPC/REST mTLS.

#### Certificates:

- CA: `ca.key`, `ca.crt` (generated at Compose startup or committed test CA)
- Vault: `vault.key`, `vault.crt` (SAN: DNS:vault)
- Guardrail: `guardrail.key`, `guardrail.crt` (SAN: DNS:guardrail)
- Payment Adapter: `adapter.key`, `adapter.crt` (SAN: DNS:upi-payment-adapter)

**Rotation (MVP):** Manual volume replacement + `docker compose up --force-recreate`.

### 7.4 Secrets Management

#### Local Dev:

- `secrets/` directory (`.gitignore`d) mounted as Docker Compose secrets
- `.env.secrets.example` template committed (no real values)
- `scripts/fetch-dev-secrets.sh` pulls from local HashiCorp Vault (dev mode) or password manager

#### CI/CD:

- GitHub Actions repository secrets → injected as Docker secrets at runtime
- OIDC to cloud secrets manager tagged [Production]

#### Test/Live Segregation (SEC-SEC-002):

- Application startup enforces `RAZORPAY_KEY_ID` starts with `rzp_test_`
- Rejection of `rzp_live_*` keys with fatal error
- CI assertion that no live credential exists in demo deployment

## 8. Observability

### 8.1 OpenTelemetry Configuration

#### SDKs:

- Python: opentelemetry-sdk + opentelemetry-instrumentation-fastapi + opentelemetry-instrumentation-psycpg2
- Node.js: @opentelemetry/sdk-node + @opentelemetry/auto-instrumentations-node

**Propagation:** W3C traceparent + baggage carrying mandate\_id and transaction\_id end-to-end.

**Root Span:** First inbound A2A Task acceptance or Human Delegation entry point.

### 8.2 Mandatory Span Attributes (NFR-OBS-002)

| Attribute                                | Type                                          | Source              |
|------------------------------------------|-----------------------------------------------|---------------------|
| constraint_hash                          | string                                        | Constraint Compiler |
| llm_invocation_id                        | string                                        | Reasoning Core      |
| payment_mandate_nonce                    | string                                        | Mandate Vault       |
| guardrail_decision                       | string (APPROVED/<br>REJECTED/<br>ESCALATED)  | Guardrail Shell     |
| grounding_status                         | string<br>(FULLY_GROUNDED/<br>PARTIAL/FAILED) | Grounding Oracle    |
| confidence_score                         | float [0.0, 1.0]                              | Confidence Gate     |
| llm.logprob_score                        | float                                         | Reasoning Core      |
| guardrail.pydantic_validation_latency_ms | float                                         | Guardrail Shell     |

### 8.3 Jaeger Deployment

```
services:
  jaeger:
    image: jaegertracing/all-in-one:1.50
    ports:
      - "16686:16686" # UI
    environment:
      - COLLECTOR_OTLP_ENABLED=true
    networks:
      - net-guardrail
      - net-ledger
```

**Trace Retention:** 7 days (MVP sufficient).

### 8.4 Decision Reconstruction (NFR-OBS-003)

**Target:** Reconstruct full decision path for any mandate in < 30 seconds.

#### Method:

1. Search Jaeger UI by mandate\_id or transaction\_id

2. Trace shows: Intent → Compilation → Negotiation → Guardrail → Vault → Payment → Settlement
  3. Each span carries `guardrail_decision`, `confidence_score`, `grounding_status`
  4. Cross-reference with `audit_events` table via `mandate_id`
- 

## 9. Deployment

### 9.1 Docker Compose Service Definitions

```
version: "3.9"

services:
  # Module 1: Constraint Compiler
  constraint-compiler:
    build: ./modules/constraint-compiler
    networks: [net-untrusted, net-guardrail]
    environment:
      - OPENAI_API_KEY_FILE=/run/secrets/openai_api_key
    secrets:
      - openai_api_key
    deploy:
      resources:
        limits: { cpus: '0.5', memory: 512M }

  # Module 3: Negotiation Engine (includes Module 2 Reasoning Core)
  negotiation-engine:
    build: ./modules/negotiation-engine
    networks: [net-llm, net-guardrail]
    environment:
      - OPENAI_API_KEY_FILE=/run/secrets/openai_api_key
      - REDIS_URL=redis://redis:6379
    secrets:
      - openai_api_key
    deploy:
      resources:
        limits: { cpus: '2.0', memory: 2048M }
    depends_on: [redis, guardrail-shell]

  # Module 4: Guardrail Shell (includes Session Manager)
  guardrail-shell:
    build: ./modules/guardrail-shell
    networks: [net-guardrail, net-signing, net-ledger]
    secrets:
      - guardrail_mtls_key
      - guardrail_mtls_cert
    deploy:
      resources:
        limits: { cpus: '1.0', memory: 1024M }
    depends_on: [postgres, redis]
```

```

# Module 5: Mandate Vault
mandate-vault:
  build: ./modules/mandate-vault
  user: "10001:10001"
  read_only: true
  cap_drop: [ALL]
  security_opt: [no-new-privileges:true]
  tmpfs: [/tmp]
  networks: [net-signing]
  secrets:
    - ap2_signing_key
    - agent_identity_key
    - vault_mtls_key
    - vault_mtls_cert
  deploy:
    resources:
      limits: { cpus: '0.5', memory: 512M }

# Module 6: UPI Payment Adapter
upi-payment-adapter:
  build: ./modules/upi-payment-adapter
  networks: [net-signing, net-settlement, net-ledger]
  secrets:
    - razorpay_test_key_id
    - razorpay_test_key_secret
    - adapter_mtls_key
    - adapter_mtls_cert
  deploy:
    resources:
      limits: { cpus: '0.5', memory: 512M }
  depends_on: [postgres]

# Module 7: Universal Commerce Adapter
universal-commerce-adapter:
  build: ./modules/universal-commerce-adapter
  networks: [net-untrusted, net-guardrail]
  secrets:
    - shopify_admin_token
  deploy:
    resources:
      limits: { cpus: '0.5', memory: 512M }

# Module 8: Ledger (PostgreSQL)
postgres:
  image: postgres:15-alpine
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./sql/init:/docker-entrypoint-initdb.d
  networks: [net-ledger]
  environment:
    - POSTGRES_USER=ledger_admin

```

```
- POSTGRES_PASSWORD_FILE=/run/secrets/postgres_password
- POSTGRES_DB=agentic_upi
secrets:
  - postgres_password
deploy:
  resources:
    limits: { cpus: '1.0', memory: 1024M }
```

# Supporting: Redis (cart locks, ephemeral state)

```
redis:
  image: redis:7-alpine
  networks: [net-guardrail]
  deploy:
    resources:
      limits: { cpus: '0.25', memory: 256M }
```

# Supporting: Jaeger

```
jaeger:
  image: jaegertracing/all-in-one:1.50
  ports: ["16686:16686"]
  networks: [net-guardrail, net-ledger]
  environment:
    - COLLECTOR_OTLP_ENABLED=true
```

# Supporting: Toxiproxy (chaos engineering)

```
toxiproxy:
  image: shopify/toxiproxy:2.5.0
  ports: ["8474:8474"]
  networks: [net-settlement, net-untrusted]
```

```
volumes:
  postgres_data:
```

```
secrets:
  openai_api_key:
    file: ./secrets/openai_api_key.txt
  ap2_signing_key:
    file: ./secrets/ap2_signing_key.pem
  agent_identity_key:
    file: ./secrets/agent_identity_key.pem
  razorpay_test_key_id:
    file: ./secrets/razorpay_test_key_id.txt
  razorpay_test_key_secret:
    file: ./secrets/razorpay_test_key_secret.txt
  shopify_admin_token:
    file: ./secrets/shopify_admin_token.txt
  postgres_password:
    file: ./secrets/postgres_password.txt
  guardrail_mtls_key:
    file: ./secrets/guardrail_mtls_key.pem
  guardrail_mtls_cert:
```

```

file: ./secrets/guardrail_mtls_cert.pem
vault_mtls_key:
  file: ./secrets/vault_mtls_key.pem
vault_mtls_cert:
  file: ./secrets/vault_mtls_cert.pem
adapter_mtls_key:
  file: ./secrets/adapter_mtls_key.pem
adapter_mtls_cert:
  file: ./secrets/adapter_mtls_cert.pem

```

## 9.2 Health Checks & Fail-Closed Behavior

```

healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8000/healthz"]
  interval: 5s
  timeout: 3s
  retries: 3
  start_period: 10s

```

**Unhealthy Dependency** → Compose restarts dependent service; Guardrail fails closed on connection refused with 503 SERVICE\_UNAVAILABLE and logs FAIL\_CLOSED\_TRIGGERED audit event.

## 10. CI/CD Pipeline

### 10.1 Test Pyramid (NFR-CI-001 / NFR-TEST-001)

| Tier                 | Scope                        | Tools                                                     | Live External Calls? |
|----------------------|------------------------------|-----------------------------------------------------------|----------------------|
| <b>Lint</b>          | Code quality                 | ruff (Python),<br>eslint (Node)                           | No                   |
| <b>Unit</b>          | Individual functions         | pytest, jest                                              | No                   |
| <b>Contract</b>      | Schema/interface conformance | schemathesis,<br>JSON Schema validation                   | No                   |
| <b>Integration</b>   | Multi-component flows        | pytest +<br>testcontainers<br>(Postgres, Redis, WireMock) | No                   |
| <b>Security Scan</b> | Vulnerability scan           | trivy (images),<br>bandit (Python)                        | No                   |
| <b>Nightly E2E</b>   | Full flow                    | pytest against<br>Razorpay Test Mode + Shopify<br>Sandbox | Yes (Test Mode only) |



## 10.2 Mocking Strategy

- **WireMock** container binds `api.razorpay.com` and `*.myshopify.com` hostnames in integration tier
- `RAZORPAY_BASE_URL=http://wiremock:8080` in test profile
- Real hostnames unreachable from unit/contract/integration jobs

## 10.3 GitHub Actions Workflow

```
name: CI
on: [push, pull_request]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps: [checkout, setup-python, setup-node, ruff, eslint]

  unit-tests:
    runs-on: ubuntu-latest
    steps: [checkout, setup-python, setup-node, pytest, jest]

  contract-tests:
    runs-on: ubuntu-latest
    steps: [checkout, setup-python, schemathesis]

  integration-tests:
    runs-on: ubuntu-latest
    steps: [checkout, setup-python, docker-compose-up-test-profile, pytest-integration]

  security-scan:
    runs-on: ubuntu-latest
    steps: [checkout, trivy, bandit]

  e2e-nightly:
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main' && github.event.schedule == '* * * * *'
    steps: [checkout, docker-compose-up, pytest-e2e-live]
    env:
      RAZORPAY_TEST_KEY_ID: ${ secrets.RAZORPAY_TEST_KEY_ID }
      RAZORPAY_TEST_KEY_SECRET: ${ secrets.RAZORPAY_TEST_KEY_SECRET }
```

---

## 11. Testing Strategy

### 11.1 Golden Dataset (FR-GRD-006 Validation)

**Composition:** 100 hand-crafted scenarios

- 40 obvious passes (within bounds, fully grounded)
- 40 obvious fails (out of bounds, hallucinated prices)
- 20 edge cases (near 0.85 threshold, partial grounding)

**Maintenance:** Co-generated by AI/LLM Engineer and System Architect during Sprint 1. Stored as tests/golden\_dataset.json.

11.2 Prompt Injection Benchmark (SEC-PI-003)

**MVP Subset:** 30 critical-path vectors across 3 highest-risk classes:

- 1. Direct Instruction Override
- 2. Homoglyph Obfuscation
- 3. Multi-hop via UCP Manifest

**CI Execution:**

- PR pipeline: 30 vectors against local quantized model (Llama-3-8B via vLLM)
- Nightly: Full 500 vectors against GPT-4o (if time permits; otherwise documented as [Production])

**Pass Criteria:** 0% ISR (Injection Success Rate) on PR subset; < 1% ISR on nightly full suite.

11.3 Graceful Failure Demo Scenario

**Selected:** Revoked Mandate Racing an In-Flight Debit (INV-004)

**Sequence:**

- 1. Agent initiates debit
- 2. Guardrail approves
- 3. Simultaneously, user triggers mandate revocation
- 4. Atomic DB lock (SELECT ... FOR UPDATE) ensures revocation wins
- 5. Payment Adapter returns 403 MANDATE\_REVOKED

**Evidence on Camera:**

- Jaeger trace showing race resolution
- Live JSONL audit log showing REVOKED timestamp before DEBIT\_ATTEMPT finalization
- HTTP 403 response with MANDATE\_REVOKED code

11.4 Chaos Experiments (Pre-Pitch)

| Experiment              | Tool                                 | Expected Behavior                                  | Metric                  |
|-------------------------|--------------------------------------|----------------------------------------------------|-------------------------|
| Razorpay timeout        | Toxiproxy (drop packets)             | Payment Adapter returns 504 GATEWAY_TIMEOUT → HITL | razorpay_timeout_count  |
| Vault network partition | iptables / Docker network disconnect | Guardrail returns 503 SERVICE_UNAVAILABLE          | vault_unreachable_count |
| Shopify webhook delay   | Toxiproxy (+10s latency)             | UCA bounded queue absorbs; no negotiation block    | uca_queue_depth         |

---

## 12. Appendices

### Appendix A: Canonical Data Schemas

#### A.1 ProposalObject (LLM → Guardrail Shell)

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "ProposalObject",
  "type": "object",
  "required": ["selected_candidate_id", "offer_price", "reasoning", "negotiation_round"],
  "properties": {
    "selected_candidate_id": { "type": "string", "format": "uuid" },
    "offer_price": { "type": "number", "minimum": 0 },
    "reasoning": { "type": "string", "maxLength": 1000 },
    "negotiation_round": { "type": "integer", "minimum": 1, "maximum": 10 }
  },
  "additionalProperties": false
}
```

#### A.2 RejectionContext (Guardrail Shell → LLM)

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "RejectionContext",
  "type": "object",
  "required": ["violated_constraint", "constraint_value", "proposed_value", "retry_count"],
  "properties": {
    "violated_constraint": {
      "type": "string",
      "enum": ["MAX_SPEND_EXCEEDED", "MERCHANT_NOT_ALLOWED", "CATEGORY_BLOCKED",
        "MONOTONICITY_VIOLATION", "SCHEMA_VIOLATION", "GROUNDING_FAILURE"]
    },
    "constraint_value": {},
    "proposed_value": {},
    "retry_count": { "type": "integer", "minimum": 0, "maximum": 3 }
  }
}
```

#### A.3 HITL\_Payload (Guardrail Shell → Human Review)

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "HITL_Payload",
  "type": "object",
  "required": ["proposal", "failed_checks", "constraint_evidence", "escalation_ttl"],
  "properties": {
    "proposal": { "$ref": "#/definitions/ProposalObject" },
    "failed_checks": { "type": "array", "items": { "type": "string" } },
    "constraint_evidence": {
```

```

    "type": "object",
    "properties": {
      "compiled_constraints_hash": { "type": "string" },
      "grounding_manifest_hash": { "type": "string" },
      "original_intent_text": { "type": "string" }
    }
  },
  "escalation_ttl": { "type": "integer", "description": "Seconds until auto-rejection; default 900" }
}

```

#### A.4 ConstraintDowngradeNotice (Protocol Bridge → LLM Context)

```

{
  "notice_type": "CONSTRAINT_DOWNGRADE",
  "original_ap2_constraint": "partial_fulfillment",
  "upi_capability": "unsupported",
  "downgrade_action": "REJECT",
  "policy_impact": "EXCLUDE_MERCHANT",
  "audit_event_id": "uuid"
}

```

## Appendix B: System Invariants Checklist

| ID      | Invariant                                                              | Verification Method                                                                 |
|---------|------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| INV-001 | LLM never holds signing keys or Razorpay credentials                   | Smoke test: filesystem + network + env inspection from Reasoning Core container     |
| INV-002 | No payment action reaches Vault/Adapter without passing Guardrail      | Code review + integration test: mock Guardrail rejection blocks Vault call          |
| INV-003 | (mandate_id, idempotency_key) executes at most once                    | DB unique constraint + concurrent load test with 10 threads                         |
| INV-004 | Revoked/expired mandate cannot be debited, even if in-flight           | pgbench concurrent debit + revocation; assert 0 successful debits post-revocation   |
| INV-005 | Every money action recorded before/atomically with execution           | Audit row timestamp    execution timestamp in all test cases                        |
| INV-006 | No single component can suppress another's audit log                   | Separate DB users per component; attempt cross-component DELETE → permission denied |
| INV-007 | AP2 constraint UPI cannot enforce → explicit downgrade/escalate/reject | Property test: random unsupported constraint → never silent drop                    |
| INV-008 | Untrusted external content can influence LLM but never authorize       | Injection test: malicious manifest → Guardrail blocks proposal                      |

Table 11 – continued

| ID      | Invariant                                               | Verification Method                                                                                  |
|---------|---------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| INV-009 | Signature/canonicalization mismatch → fail closed       | Negative crypto test suite: altered byte → rejection                                                 |
| INV-010 | Exposure bounds enforced in Policy Engine code, not LLM | Unit test: Policy Engine rejects proposal exceeding hard-coded max_spend regardless of LLM reasoning |

Appendix C: Production Evolution Roadmap

| MVP Limitation                               | Production Target                                |
|----------------------------------------------|--------------------------------------------------|
| Software-backed keys (documented limitation) | HSM/KMS (AWS KMS, HashiCorp Vault)               |
| Docker Compose on single host                | Kubernetes with gVisor sandboxed containers      |
| REST + JSON inter-service                    | gRPC + Protobuf (same canonical payload)         |
| PostgreSQL outbox + polling                  | Kafka outbox + consumer groups                   |
| APScheduler in-process TTLs                  | Redis Streams / Kafka for distributed scheduling |
| 30-vector injection benchmark                | 500-vector suite across 7 classes                |
| GitHub Actions repository secrets            | OIDC + cloud secrets manager                     |
| Manual certificate rotation                  | Automated short-lived certs (cert-manager)       |
| Jaeger all-in-one                            | Full OTel Collector + Prometheus/Grafana stack   |
| Single Shopify REST integration              | Multi-platform GraphQL facade                    |

End of Software Design Document